

# BANCO DE DADOS RELACIONAL

**TRIGGERS - Automatizando regras de negócio no banco de dados**

***Professora:***

Lucineide Pimenta

# Objetivos da aula



- ❑ Entender o que é uma Trigger
- ❑ Criar triggers no PostgreSQL
- ❑ Automatizar ações no banco de dados
- ❑ Compreender BEFORE e AFTER
- ❑ Relacionar triggers com cenários reais
- ❑ Refletir sobre vantagens e riscos do uso excessivo

# Contextualização

- ❑ Imagine uma biblioteca onde:
  - ❑ um livro é emprestado;
  - ❑ mas o estoque não é atualizado;
  - ❑ ou um livro é removido e ninguém registra isso;
  - ❑ ou um aluno devolve um livro atrasado e o sistema não gera multa.
- ❑ Tudo isso depende de regras automáticas.

# O que são triggers?

- ❑ Uma **trigger** é um recurso do banco que **executa automaticamente** um comando quando ocorre um evento em uma tabela.

## ***Definição simples***

Uma Trigger é um mecanismo automático executado pelo banco de dados quando um evento acontece.

## Sensor automático

- ❑ Uma trigger funciona como:
  - ❑ sensor de presença;
  - ❑ porta automática;
  - ❑ alarme;
  - ❑ detector de fumaça.
- ❑ Quando algo acontece:
  - ❑ a trigger reage automaticamente.

# Eventos que disparam triggers

| <b><i>Evento</i></b> | <b><i>Quando dispara</i></b> | <b><i>Exemplo</i></b> |
|----------------------|------------------------------|-----------------------|
| INSERT               | Ao inserir                   | novo empréstimo       |
| UPDATE               | Ao atualizar                 | alteração de estoque  |
| DELETE               | Ao deletar                   | exclusão de livro     |

# Tipos de triggers no PostgreSQL

## (1) BEFORE

- ❑ Executa **antes** da ação acontecer.  
Serve para:
  - ❑ Validar dados
  - ❑ Bloquear operações
  - ❑ Ajustar valores antes de inserir

## (2) AFTER

- ❑ Executa depois que a ação ocorreu.  
Serve para:
  - ❑ Criar logs
  - ❑ Sincronizar dados
  - ❑ Criar histórico

## (1) BEFORE

- ❑ *"Antes de salvar, valide."*
- ❑ Exemplo:
  - ❑ impedir número de páginas negativas.

## (2) AFTER

- ❑ *"Depois de salvar, execute algo."*
- ❑ Exemplo:
  - ❑ registrar log;
  - ❑ atualizar estoque;
  - ❑ criar histórico.



# Estrutura de uma Trigger

## ❑ Etapa 1 — criar FUNCTION

```
CREATE OR REPLACE FUNCTION nome_funcao()  
RETURNS TRIGGER  
AS $$  
BEGIN  
  
    -- lógica  
  
    RETURN NEW;  
  
END;  
$$ LANGUAGE plpgsql;
```

# Estrutura de uma Trigger

## ❑ Etapa 2 — criar TRIGGER

```
CREATE TRIGGER nome_trigger  
AFTER INSERT  
ON tabela  
FOR EACH ROW  
EXECUTE FUNCTION nome_funcao();
```

# PRIMEIRA TRIGGER

**--Adicionar coluna quantidade**

*ALTER TABLE livro*

*ADD COLUMN quantidade INTEGER DEFAULT 5;*

# PRIMEIRA TRIGGER

## -- **Atualizar estoque automaticamente**

- Sempre que ocorrer um empréstimo:
  - diminuir automaticamente a quantidade do livro.

### □ **Passo 1 – criar FUNCTION**

```
CREATE OR REPLACE FUNCTION  
reduzir_estoque()  
RETURNS TRIGGER  
AS $$  
BEGIN
```

```
UPDATE livro  
SET quantidade = quantidade - 1  
WHERE id_livro = NEW.id_livro;
```

```
RETURN NEW;
```

```
END;  
$$ LANGUAGE plpgsql;
```

# PRIMEIRA TRIGGER

*A linha:*

*NEW.id\_livro*

- ❑ Representa:
  - ❑ o valor inserido na nova linha.

# PRIMEIRA TRIGGER

## Passo 2 – criar Trigger

```
CREATE TRIGGER trg_reduzir_estoque  
AFTER INSERT  
ON emprestimo  
FOR EACH ROW  
EXECUTE FUNCTION reduzir_estoque();
```

# PRIMEIRA TRIGGER

## Teste

### *--Criar empréstimo*

```
INSERT INTO emprestimo  
(id_livro, data_emprestimo)  
VALUES  
(1, CURRENT_DATE);
```

### *--Associar livro ao empréstimo*

```
INSERT INTO  
emprestimo_livro(id_emprestimo,  
id_livro)VALUES(2, 1);
```

### *--Verificação*

```
SELECT titulo, quantidade  
FROM livro;
```

- >  Rules
  - ✓  Triggers (1)
    - ✓  trg\_reduzir\_estoque
      -  reduzir\_estoque()
-  livro
-  Trigger Functions (1)
  -  reduzir\_estoque()
-  Types
-  Views
- scriptions
- \_alerta
- sts
- alogs
- nt Triggers
- ensions
- eign Data Wrappers
- iguages



# SEGUNDA TRIGGER

- ❑ Impedir páginas negativas
- ❑ **Problema real**
- ❑ Usuário tenta cadastrar:
  - ❑ páginas negativas;
  - ❑ estoque negativo;
  - ❑ dados inválidos.

# SEGUNDA TRIGGER

## ❑ BEFORE TRIGGER

### Function

```
CREATE OR REPLACE FUNCTION  
validar_paginas()  
RETURNS TRIGGER  
AS $$  
BEGIN
```

```
IF NEW.num_paginas <= 0 THEN  
    RAISE EXCEPTION  
    'Número de páginas inválido.';  
END IF;  
  
RETURN NEW;  
  
END;  
$$ LANGUAGE plpgsql;
```

# SEGUNDA TRIGGER

## TRIGGER

```
CREATE TRIGGER trg_validar_paginas  
BEFORE INSERT OR UPDATE  
ON livro  
FOR EACH ROW  
EXECUTE FUNCTION validar_paginas();
```

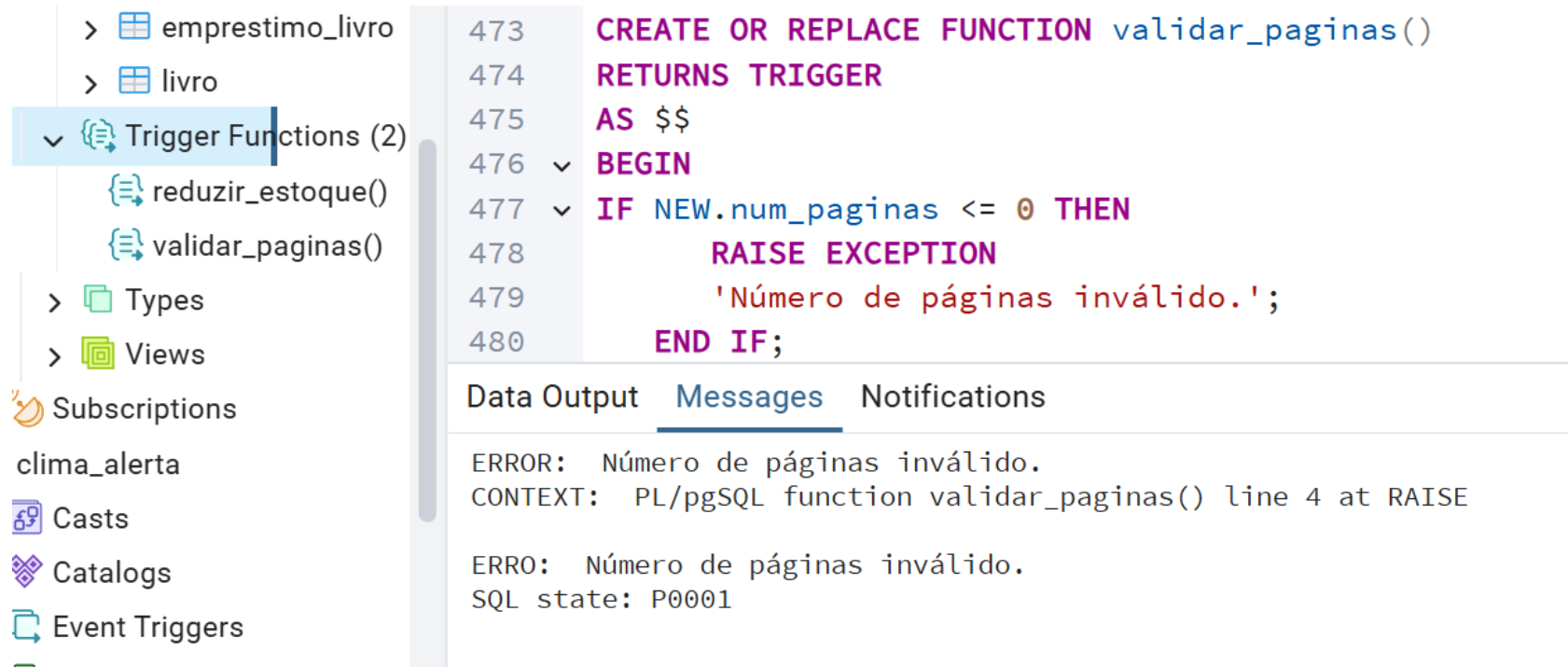
## Teste

```
INSERT INTO livro  
(titulo, num_paginas, quantidade, id_autor)  
VALUES  
('Livro Inválido', -50, 2, 1);
```

# SEGUNDA TRIGGER

## ❑ Resultado esperado

- ❑ ERRO: Número de páginas inválido.



The screenshot displays a database management interface. On the left, a tree view shows the database structure with folders for 'emprestimo\_livro', 'livro', 'Trigger Functions (2)', 'Types', 'Views', 'Subscriptions', 'casts', 'Catalogs', and 'Event Triggers'. Under 'Trigger Functions (2)', two functions are listed: 'reduzir\_estoque()' and 'validar\_paginas()'. The 'validar\_paginas()' function is selected, and its code is displayed in the main pane. The code is as follows:

```
473 CREATE OR REPLACE FUNCTION validar_paginas()  
474 RETURNS TRIGGER  
475 AS $$  
476 BEGIN  
477 IF NEW.num_paginas <= 0 THEN  
478     RAISE EXCEPTION  
479     'Número de páginas inválido.';  
480 END IF;
```

Below the code, there are three tabs: 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, showing the following error message:

```
ERROR: Número de páginas inválido.  
CONTEXT: PL/pgSQL function validar_paginas() line 4 at RAISE
```

Below this, another error message is shown:

```
ERRO: Número de páginas inválido.  
SQL state: P0001
```

# Importante

- ❑ A trigger protege o banco mesmo que:
  - ❑ frontend falhe;
  - ❑ backend tenha erro;
  - ❑ alguém execute SQL manualmente.

# TERCEIRA TRIGGER

- ❑ Criar LOG automático
- ❑ **Problema real**
  - ❑ Quem alterou o estoque?
  - ❑ Quando?
  - ❑ Qual livro foi alterado?

# TERCEIRA TRIGGER

## ❑ Tabela de log

```
CREATE TABLE log_livro (  
    id_log SERIAL PRIMARY KEY,  
    titulo VARCHAR(100),  
    quantidade_antiga INTEGER,  
    quantidade_nova INTEGER,  
    data_log TIMESTAMP  
);
```

# TERCEIRA TRIGGER

## ❑ Function

*CREATE OR REPLACE FUNCTION registrar\_log()*

*RETURNS TRIGGER*

*AS \$\$*

*BEGIN*

*INSERT INTO log\_livro*

*(*

*titulo,*

*quantidade\_antiga,*

*quantidade\_nova,*

*data\_log*

*)*

*VALUES*

*(*

*OLD.titulo,*

*OLD.quantidade,*

*NEW.quantidade,*

*CURRENT\_TIMESTAMP*

*);*

*RETURN NEW;*

*END;*

*\$\$ LANGUAGE plpgsql;*



# TERCEIRA TRIGGER

## ❑ Explicação

| Palavra | Significado  |
|---------|--------------|
| OLD     | valor antigo |
| NEW     | valor novo   |

# TERCEIRA TRIGGER

## ❑ **Trigger**

```
CREATE TRIGGER trg_log_estoque  
AFTER UPDATE  
ON livro  
FOR EACH ROW  
EXECUTE FUNCTION registrar_log();
```

## ❑ **Teste**

```
UPDATE livro  
SET quantidade = 10  
WHERE id_livro = 1;
```

## **Verificar log**

```
SELECT * FROM log_livro;
```

# TERCEIRA TRIGGER

## ❑ Resultado esperado

The screenshot displays a database management interface. On the left, a tree view shows the database structure with 'livro' expanded, revealing three trigger functions: 'reduzir\_estoque()', 'registrar\_log()', and 'validar\_paginas()'. The main editor shows a SQL script for the 'validar\_paginas()' trigger, which includes a comment '--Verificar log' and a query 'SELECT \* FROM log\_livro;'. Below the editor, the 'Data Output' tab is active, showing a single row of data from the 'log\_livro' table. The row contains the following values: id\_log (1), titulo ('O Senhor dos Anéis'), quantidade\_antiga (4), quantidade\_nova (10), and data\_log (2026-05-26 18:23:05.989805).

```
549
550 --Verificar log
551 SELECT * FROM log_livro;
552
553
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

|   | id_log<br>[PK] integer | titulo<br>character varying (100) | quantidade_antiga<br>integer | quantidade_nova<br>integer | data_log<br>timestamp without time zone |
|---|------------------------|-----------------------------------|------------------------------|----------------------------|-----------------------------------------|
| 1 | 1                      | O Senhor dos Anéis                | 4                            | 10                         | 2026-05-26 18:23:05.989805              |

# Situações REAIS de uso de Triggers

| Sistema    | Trigger                        |
|------------|--------------------------------|
| Banco      | registrar movimentação         |
| E-commerce | atualizar estoque              |
| Hospital   | histórico médico               |
| Escola     | gerar frequência               |
| RH         | registrar alterações salariais |
| Segurança  | auditoria                      |

# Vantagens das Triggers

| Vantagem      | Explicação                   |
|---------------|------------------------------|
| Automação     | elimina tarefas manuais      |
| Segurança     | protege integridade          |
| Centralização | regra fica no banco          |
| Auditoria     | gera histórico automático    |
| Padronização  | qualquer sistema segue regra |

# Desvantagens e Limitações

| Problema             | Explicação                             |
|----------------------|----------------------------------------|
| Complexidade         | difícil manutenção                     |
| Processamento oculto | ação automática pode confundir         |
| Performance          | excesso de trigger deixa sistema lento |
| Dependência          | lógica escondida no banco              |
| Debug difícil        | erros podem ser difíceis de rastrear   |

# Quando usar Trigger?

- ❑ **Use quando:**
  - ❑ regra precisa ser automática;
  - ❑ integridade é crítica;
  - ❑ auditoria é importante;
  - ❑ múltiplos sistemas acessam o banco.
- ❑ **Evite quando:**
  - ❑ regra pertence ao frontend;
  - ❑ processamento é muito pesado;
  - ❑ lógica muda frequentemente.

# Quando usar triggers?

## ❑ Use triggers quando:

- ❑ Precisa garantir integridade além das constraints
- ❑ Precisa manter logs automáticos
- ❑ Precisa completar dados automaticamente
- ❑ Precisa validar elementos que dependem de várias tabelas

## ❑ Evite triggers quando:

- ❑ Está tentando substituir regras do sistema
- ❑ A lógica fica complexa demais
- ❑ Pode ser feito facilmente na aplicação
- ❑ Pode gerar dependências circulares



# Importante

- ❑ **Erro comum em projetos**
- ❑ Desenvolvedores normalmente:
  - ❑ colocam tudo no frontend;
  - ❑ não persistem corretamente;
  - ❑ deixam regras críticas apenas no React/Node.
- ❑ Se alguém acessar o banco diretamente:
  - ❑ as regras deixam de existir.

# Boas Práticas com Triggers

- ✓ Usar triggers **somente quando a lógica pertence ao BD.**
- ✓ Documentar (comentários e README).
- ✓ **Nomear padrão:** *trg\_acao\_tabela*.
- ✓ Criar função separada da trigger.
- ✓ Manter triggers simples (não coloque regras demais nelas).
- ✓ Evitar triggers que alteram outras tabelas sem necessidade.
- ✓ Evitar lógica de negócio complexa: isso é da aplicação.

# Boas Práticas com Triggers

- ❑ **Regra crítica**
  - ❑ no banco.
- ❑ **Regra visual**
  - ❑ frontend.
- ❑ **Regra de negócio complexa**
  - ❑ backend e banco.

# Por que usar triggers?

- ❑ **Triggers são úteis para:**
  - ❑ Garantir integridade
  - ❑ Manter registros históricos (auditoria)
  - ❑ Criar logs
  - ❑ Validar dados antes de salvar
  - ❑ Ajustar ou completar informações automaticamente
  - ❑ Evitar duplicação de lógica em várias partes da aplicação

# Desvantagens e Limitações de Triggers

- ❑ Dificilmente visíveis: "lógica escondida".
- ❑ Difícil depurar (debug).
- ❑ Podem impactar performance.
- ❑ Podem criar dependências difíceis de manter.
- ❑ Não substituem validação da aplicação

# O que veremos na próxima aula

- **TRIGGERS**
  - Triggers compostas,
  - múltiplos eventos,
  - triggers em cascata e
  - anti-patterns.



# Referências Bibliográfica da Aula

## Livros:

**Elmasri & Navathe (2010).** Sistemas de Banco de Dados.

**Silberschatz et al. (2011).** Sistemas de Banco de Dados.

## Links úteis:

 [PostgreSQL Docs](#)

 [DBDiagram.io](#)

# Bibliografia Básica

- ❑ DATE, C. J. **Introdução a sistemas de bancos de dados**. Rio de Janeiro, Elsevier: Campus, 2004.
- ❑ ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**. 7 ed. São Paulo: Pearson, 2018.
- ❑ SILBERSCHATZ, A.; SUNDARSHAN, S.; KORTH, H. F. **Sistema de banco de dados**. Rio de Janeiro: Elsevier Brasil, 2016.



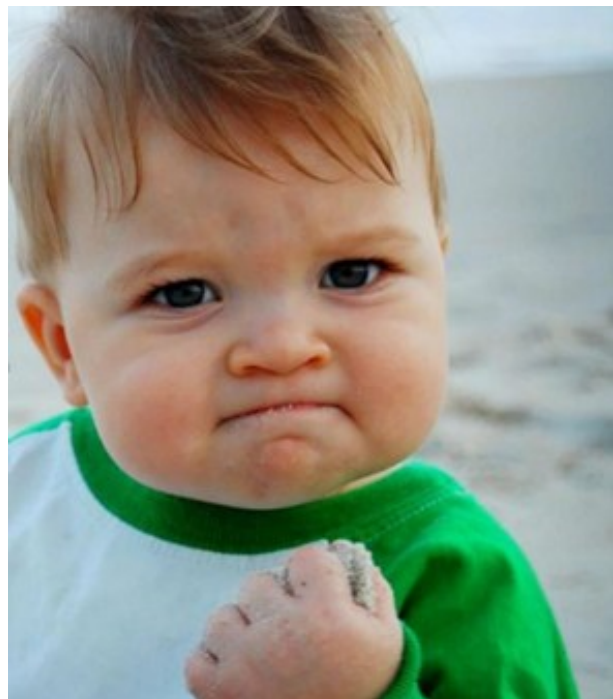
# Bibliografia Complementar

- ❑ BEAULIEU, A. **Aprendendo SQL**. São Paulo: Novatec, 2010.
- ❑ GILLENSON, M. L. **Fundamentos de Sistemas de Gerência de Banco de Dados**. Rio de Janeiro: LTC, 2006.
- ❑ MACHADO, F. N. R. **Banco de Dados: Projeto e Implementação**. São Paulo: Érica, 2005.
- ❑ OTEY, M; OTEY, D. **Microsoft SQL Server 2005: Guia do Desenvolvedor**. Rio de Janeiro: Ciência Moderna, 2007.
- ❑ RAMAKRISHNAN, R.; GEHRKE, J. **Sistemas de Gerenciamento de Bancos de Dados**. 3 ed. Porto Alegre: Bookman, 2008.
- ❑ ROB, P; CORONEL, C. **Sistemas de Banco de Dados: Projeto, Implementação e Gerenciamento**. 8 ed. São Paulo: Cengage Learning, 2011.
- ❑ TEOREY, T; LIGHTSTONE, S; NADEAU, T. **Projeto e Modelagem de Bancos de Dados**. São Paulo: Campus, 2006.

# Dúvidas?



# Considerações Finais



**Professora:  
Lucineide Pimenta**

**Bom descanso à todos!**

